

Marketplace Microservices

Clear & Precise Technical Specification

Version: MVP Architecture — 7 Services

1. System Purpose

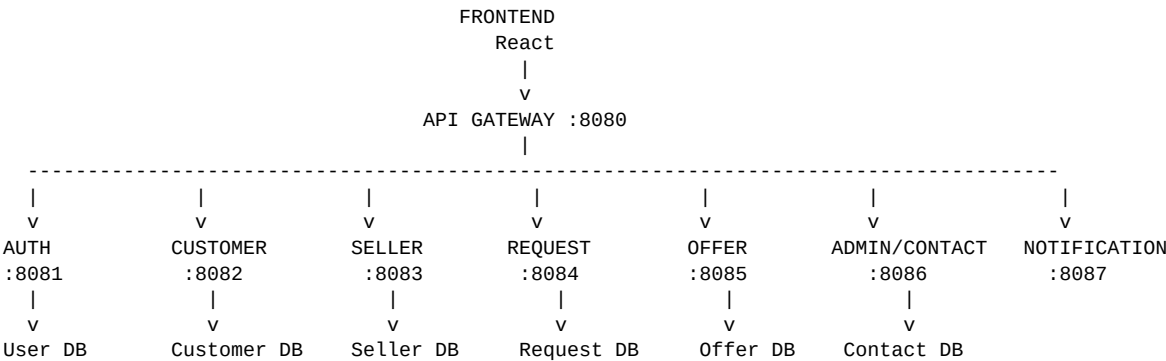
The platform groups customers who want to buy the same item. A customer can create a new purchase request or join an existing one. Sellers can see the total demand and submit an offer. An administrator decides whether an offer is approved. After approval, the system grants the seller permission to access the phone numbers of the relevant customers.

Important: There is no payment or order service in this MVP. The system stops after an authorized seller receives customer contact information.

2. The Exact Business Rules

Rule	Definition
R1	A customer can create a purchase request.
R2	A customer can join an existing OPEN request for the same item.
R3	Each participant specifies the quantity they want.
R4	The Request Service calculates totalCustomers and totalQuantity.
R5	A seller can submit an offer only for an existing request.
R6	A new offer starts with status PENDING.
R7	Only an ADMIN can approve or reject an offer.
R8	Approving an offer does not automatically expose every phone number without creating contact permission.
R9	A seller can see a phone number only when ContactAccess is GRANTED for that seller and request.
R10	Phone numbers are owned by User/Auth Service and are never stored in Request or Offer Service.

3. Final Architecture



Notification DB -> SMS / Email / Push Provider

4. Service Boundaries — Who Owns What?

Service	Owns	Must NOT Own
User/Auth	User account, email, password hash, role, phone number, JWT/AccessToken	Customer profile, seller store, request, offer
Customer	Customer profile linked to userId	Password, authentication credentials

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate and return tokens
POST	/api/auth/refresh	Refresh access token
POST	/api/auth/logout	End session
GET	/api/users/me	Get authenticated user
PUT	/api/users/me	Update own account

Internal API

Method	Endpoint	Purpose
GET	/internal/users/{userId}	Get basic internal identity data
GET	/internal/users/{userId}/phone	Get phone number — restricted

8. Customer Service

Port: 8082 | Database: customer_db

Stores customer-specific profile data. It does not authenticate users.

Main Data

Customer

- customerId
- userId (unique)
- firstName
- lastName
- profileStatus
- createdAt
- updatedAt

Public API

Method	Endpoint	Purpose
POST	/api/customers	Create customer profile
GET	/api/customers/me	Get own profile
PUT	/api/customers/me	Update own profile

Internal API

Method	Endpoint	Purpose
GET	/internal/customers/by-user/{userId}	Resolve customerId
GET	/internal/customers/{customerId}	Read customer profile

9. Seller Service

Port: 8083 | Database: seller_db

Stores seller/store information. Seller registration does not require manual verification in this MVP.

Main Data

Seller

- sellerId
- userId (unique)
- storeName

- description
- city
- address
- rating
- createdAt
- updatedAt

Public API

Method	Endpoint	Purpose
POST	/api/sellers	Create seller profile
GET	/api/sellers/me	Get own seller profile
PUT	/api/sellers/me	Update own seller profile
GET	/api/sellers/{sellerId}	Get public seller profile

Internal API

Method	Endpoint	Purpose
GET	/internal/sellers/by-user/{userId}	Resolve sellerId
GET	/internal/sellers/{sellerId}	Read seller profile

10. Request Service

Port: 8084 | Database: request_db

Manages demand. A request represents one item that multiple customers want to buy.

Main Data

PurchaseRequest

- requestId
- itemName
- description
- category
- status: OPEN | OFFER_PENDING | OFFER_APPROVED | CLOSED | CANCELLED
- totalCustomers
- totalQuantity
- createdAt
- updatedAt

RequestParticipant

- participantId
- requestId
- customerId
- quantity
- joinedAt

Public API

Method	Endpoint	Purpose
POST	/api/requests	Create a new request
GET	/api/requests	Search/list requests
GET	/api/requests/{requestId}	Get request and demand
POST	/api/requests/{requestId}/participants	Join request
PUT	/api/requests/{requestId}/participants/me	Change own quantity
DELETE	/api/requests/{requestId}/participants/me	Leave request
GET	/api/requests/me	Get requests joined by current customer

Internal API

Method	Endpoint	Purpose
GET	/internal/requests/{requestId}	Validate/read request
GET	/internal/requests/{requestId}/demand	Read totalCustomers + totalQuantity
GET	/internal/requests/{requestId}/participants	Get participant customerIds

11. Offer Service

Port: 8085 | Database: offer_db

Stores seller offers. An offer is a proposal to satisfy a specific purchase request.

Main Data

Offer

- offerId
- sellerId
- requestId
- availableQuantity
- pricePerUnit
- currency
- description
- status: PENDING | APPROVED | REJECTED | CANCELLED
- createdAt
- updatedAt

Public API

Method	Endpoint	Purpose
POST	/api/offers	Submit offer
GET	/api/offers/{offerId}	Get offer
GET	/api/offers/request/{requestId}	Get offers for request
GET	/api/offers/me	Get own seller offers
PUT	/api/offers/{offerId}	Update own PENDING offer
DELETE	/api/offers/{offerId}	Cancel own offer

Internal API

Method	Endpoint	Purpose
GET	/internal/offers/pending	List offers awaiting admin review
PATCH	/internal/offers/{offerId}/status	Set APPROVED or REJECTED

12. Admin/Contact Service

Port: 8086 | Database: admin_contact_db

Approves or rejects offers and manages permission to expose customer contact information.

Main Data

OfferDecision

- decisionId
- offerId
- adminUserId
- decision: APPROVED | REJECTED
- reason
- decidedAt

ContactAccess

- accessId
- sellerId
- customerId
- requestId
- offerId
- status: GRANTED | REVOKED | EXPIRED
- grantedBy
- grantedAt
- expiresAt

Public API

Method	Endpoint	Purpose
GET	/api/admin/offers/pending	List offers for review
POST	/api/admin/offers/{offerId}/approve	Approve offer
POST	/api/admin/offers/{offerId}/reject	Reject offer
GET	/api/admin/contact-access	List access permissions
DELETE	/api/admin/contact-access/{accessId}	Revoke permission
GET	/api/contacts/requests/{requestId}	Seller retrieves authorized contacts

Internal API

Method	Endpoint	Purpose
GET	/internal/contact-access	Validate seller/customer contact permission

13. Notification Service

Port: 8087 | Database: notification_db

Sends and records notifications. It is not the source of truth for business data.

Main Data

Notification

- notificationId
- userId
- type
- channel: IN_APP | EMAIL | SMS | PUSH
- title
- message
- status: PENDING | SENT | FAILED | READ
- createdAt
- sentAt

Examples:

REQUEST_JOINED

NEW_OFFER

OFFER_APPROVED

OFFER_REJECTED

CONTACT_ACCESS_GRANTED

REQUEST_CLOSED

Public API

Method	Endpoint	Purpose
GET	/api/notifications/me	Get own notifications
GET	/api/notifications/me/unread-count	Get unread count
PATCH	/api/notifications/{notificationId}/read	Mark own notification as read

Internal API

Method	Endpoint	Purpose
POST	/internal/notifications	Create/send one notification
POST	/internal/notifications/bulk	Create/send multiple notifications

14. Exact Interaction Flow 1 — Create or Join Request

STEP 1

Customer -> API Gateway -> POST /api/requests

STEP 2

Request Service reads userId from JWT.

STEP 3

Request Service -> Customer Service:
GET /internal/customers/by-user/{userId}

STEP 4

Customer Service returns customerId.

STEP 5

Request Service creates a new PurchaseRequest
OR finds an existing matching OPEN request.

STEP 6

Request Service creates RequestParticipant.

STEP 7

Request Service updates:
- totalCustomers
- totalQuantity

STEP 8

Request Service -> Notification Service:
POST /internal/notifications
type = REQUEST_JOINED

15. Exact Interaction Flow 2 — Seller Creates Offer

STEP 1

Seller -> API Gateway -> POST /api/offers

STEP 2

Offer Service reads userId from JWT.

STEP 3

Offer Service -> Seller Service:
GET /internal/sellers/by-user/{userId}

STEP 4

Seller Service returns sellerId.

STEP 5

Offer Service -> Request Service:
GET /internal/requests/{requestId}

STEP 6

If request exists and accepts offers:
create Offer with status = PENDING.

STEP 7

Offer Service -> Notification Service:
NEW_OFFER notification to ADMIN users.

16. Exact Interaction Flow 3 — Admin Approves Offer

STEP 1

Admin -> POST /api/admin/offers/{offerId}/approve

STEP 2

Admin/Contact Service verifies role = ADMIN.

STEP 3

Admin/Contact Service -> Offer Service:
PATCH /internal/offers/{offerId}/status
status = APPROVED

STEP 4

Admin/Contact Service -> Request Service:
GET /internal/requests/{requestId}/participants

STEP 5

Request Service returns participant customerIds.

STEP 6

Admin/Contact Service creates one ContactAccess record
for every customer who should be exposed to the seller.

STEP 7

Admin/Contact Service -> Notification Service:
- OFFER_APPROVED to seller
- CONTACT_ACCESS_GRANTED to seller

17. Exact Interaction Flow 4 — Seller Gets Phone Numbers

STEP 1

Seller -> GET /api/contacts/requests/{requestId}

STEP 2

Admin/Contact Service identifies seller from JWT.

STEP 3

Admin/Contact Service verifies:
- seller has an APPROVED offer for requestId
- ContactAccess.status = GRANTED

STEP 4

For each authorized customer:
Admin/Contact Service needs userId.

It resolves:

customerId -> Customer Service -> userId

STEP 5

Admin/Contact Service -> User/Auth Service:
GET /internal/users/{userId}/phone

STEP 6

User/Auth Service returns phone number only for the
authorized internal request.

STEP 7

Admin/Contact Service returns only the authorized contacts
to the seller.

Important: The Request Service and Offer Service never return phone numbers. All contact access is controlled by Admin/Contact Service.

18. Notification Events

Event	Produced By	Recipient
REQUEST_JOINED	Request Service	Customer
NEW_OFFER	Offer Service	Admin
OFFER_APPROVED	Admin/Contact Service	Seller
OFFER_REJECTED	Admin/Contact Service	Seller
CONTACT_ACCESS_GRANTED	Admin/Contact Service	Seller
REQUEST_CLOSED	Request Service	All request participants

19. API Gateway Routes

Public Route	Destination
/api/auth/**	User/Auth Service :8081
/api/users/**	User/Auth Service :8081
/api/customers/**	Customer Service :8082
/api/sellers/**	Seller Service :8083
/api/requests/**	Request Service :8084
/api/offers/**	Offer Service :8085
/api/admin/**	Admin/Contact Service :8086
/api/contacts/**	Admin/Contact Service :8086
/api/notifications/**	Notification Service :8087

20. Security Model

JWT:

Authorization: Bearer <access-token>

Recommended claims:

```
{
  "sub": "userId",
  "role": "CUSTOMER | SELLER | ADMIN"
}
```

Access rules:

CUSTOMER -> customer + request APIs

SELLER -> seller + offer + authorized contact APIs

ADMIN -> admin approval APIs

INTERNAL -> service-to-service only

21. Final Responsibility Summary

Service	One Clear Responsibility
User/Auth	Identity, authentication, roles and phone numbers
Customer	Customer profile
Seller	Seller/store profile
Request	What customers want and how much total demand exists
Offer	What a seller can provide, quantity, price and offer state
Admin/Contact	Approve/reject offers and authorize contact access
Notification	Inform users and track notification delivery

This architecture keeps the responsibilities separated and makes the sensitive phone-number flow explicit: **Offer approved** → **ContactAccess granted** → **Seller requests contacts** → **permission verified** → **phone number retrieved from User/Auth Service**.